OMNISCIA

November 11th, 2026

Smart Contract Audit Report

Term Finance

Gnosis Infrastructure Deployment Validation

WEB	omniscia.io
EMAIL	info@omniscia.io
ONLINE REPORT	https://omniscia.io/reports/term-finance-gnosis-infrastructure-deployment-validation-6acdf0fa9f2369c00b5d6c0e

OMNISCIA

ABOUT OMNISCIA

Omniscia.io is one of the fastest growing and most trusted blockchain security firms and has rapidly become a true market leader. To date, our team has collectively secured over 370+ clients, detecting 1,500+ high-severity issues in widely adopted smart contracts.

Founded in France at the start of 2020, and with a track record spanning back to 2017, our team has been at the forefront of auditing smart contracts, providing expert analysis and identifying potential vulnerabilities to ensure the highest level of security of popular smart contracts, as well as complex and sophisticated decentralized protocols.

Our clients, ecosystem partners, and backers include leading ecosystem players such as L'Oréal, Polygon, AvaLabs, Gnosis, Morpho, Vesta, Gravita, Olympus DAO, Fetch.ai, and LimitBreak, among others.

To keep up to date with all the latest news and announcements follow us on twitter @omniscia_sec.

WEB	omniscia.io
EMAIL	info@omniscia.io
ONLINE REPORT	https://omniscia.io/reports/term-finance-gnosis-infrastructure-deployment-validation-6acdf0fa9f2369c00b5d6c0e

SECURITY AUDIT REPORT

Gnosis Infrastructure Deployment Validation Security Audit

The audit encompasses three unique Gnosis guards as well as deployment validation of the system.

FINDINGS	ADDRESSED	HIGHEST SEVERITY	REVISIONS
15	14	Medium	3

Audit Report Revisions

INITIAL REVIEW October 14th, 2026

COMMIT `b78b57ea6e` ❖ `eb4cb747d2`

15 identified

REMEDIATION 01 November 4th, 2026

COMMIT `d29979f747` ❖ `0c4d95f43d`

11 alleviated · 2 partially alleviated · 1 nullified · 1 not alleviated

FINAL November 11th, 2026

COMMIT `69e22d825b` ❖ `0a0d41ab14`

2 alleviated · 1 acknowledged

Audit Overview

We were tasked with performing an audit of the Term Finance codebase and in particular their custom Gnosis Safe guards.

In addition to the security audit, we were tasked with validating that the deployment of the various Gnosis Safe implementations and supplementary governance-related infrastructure of the Term Finance system is secure and adheres to the documented procedures as well as configurations.

Over the course of the audit, we identified certain governance pathways that weaken the expected security guarantees of the system alongside issues related to the documentation of the system as well as its deployment.

For a more detailed overview of the deployment validation approach, please consult the relevant dedicated chapter contained within the audit report.

→ **We advise the Term Finance team to closely evaluate all minor-and-above findings identified in the report and promptly remediate them as well as consider all optimizational exhibits identified in the report.**

Post-Audit Conclusions

CONCLUSION 01 `d29979f747`

The Term Finance team iterated through all findings within the report and provided us with a revised commit hash to evaluate all exhibits on.

We evaluated all alleviations performed by Term Finance and have identified that certain exhibits have not been adequately dealt with. We advise the Term Finance team to revisit the following exhibits: `PGD-02M`, `PGD-03M`, `TTN-01M`

CONCLUSION 02 `69e22d825b`

The Term Finance team provided us access to private repositories covering changes to the `TermToken` implementation as well as to their internal operational runbooks.

Additionally, an upgrade was performed for the `TermToken` specifically on block `26,358,239` to point to the upgraded implementation, ensuring the blockchain state reflects the changes that arose from this audit on this contract.

Exhibit `PGD-02M` was addressed in the runbooks directly, whereas `PGD-03M` was addressed in the runbooks and its governance-related recommendation was acknowledged by the Term Finance team.

As a result, we consider all outputs of the audit report properly consumed by the Term Finance team with no further remediative actions remaining.

Audit Synopsis

SEVERITY	IDENT.	ALLEV.	PARTIAL	ACKN.	NULLIF.
Major	0	0	0	0	0
Medium	1	1	0	0	0
Minor	5	4	0	1	0
Informational	9	8	0	0	1
Unknown	0	0	0	0	0

13 alleviated · 1 acknowledged · 1 nullified

During the audit, we filtered and validated a total of **4 findings utilizing static analysis** tools as well as identified a total of **11 findings during the manual review** of the codebase. We strongly recommend that any minor severity or higher findings are dealt with promptly prior to the project's launch as they can introduce potential misbehaviours of the system as well as exploits.

Report Chapters

The list below covers each segment of the audit in depth and links to the respective chapter of the report:

🎯 Scope →

A specification of the audit scope, including metadata such as the project's language and deployed network.

💻 Compilation →

Includes the compilation instructions that were followed as well as any issues that were identified by the compiler itself.

🔍 Static Analysis →

Briefly describes the output of the static analyzers of the project.

👁 Manual Review →

Briefly describes the output of the manual review of the project.

🖌 Code Style →

Describes the dedicated section of coding style and optimizational findings of the audit

📄 Deployment Validation of Gnosis Safe Infrastructure →

A dedicated chapter that outlines the deployment validation steps our team undertook to cross-reference the documents provided to us by the Term Finance team with the live blockchain state.

CHAPTER 01

Scope

The audit engagement encompassed a specific list of contracts that were present in the commit hash of the repository that was in scope. The tables below detail certain meta-data about the target of the security assessment and a navigation chart is present at the end that links to the relevant findings per file.

Target

REPOSITORY	term-finance/zodiac-modifier-roles-v1
COMMIT	b78b57ea6e37c3eee48e265acb3c3dca4f76d5ff
LANGUAGE	Solidity
INTEGRATIONS	open-zeppelin
REVISIONS	b78b57ea6e, d29979f747, 69e22d825b

Findings Per File

ConfigLockGuard.sol — CLG-S CLG-M CLG-C — 8 →

SetTxNonceGuard.sol — STN-S — 2 →

PauseGuard.sol — PGD-M PGD-C — 4 →

TermToken.sol — TTN-M — 1 →

Contracts Assessed

☐ Carries findings ☐ No findings

CLG ConfigLockGuard.sol 8	PGD PauseGuard.sol 4	STN SetTxNonceGuard.sol 2
TTN TermToken.sol 1		

CHAPTER 02

Compilation

Includes the compilation instructions that were followed as well as any issues that were identified by the compiler itself.

PIPELINE	SOLC VERSION	COMPILER ERRORS
hardhat	0.8.17	None

The project utilizes `hardhat` as its development pipeline tool, containing an array of tests and scripts coded in TypeScript.

To compile the project, the `compile` command needs to be issued via the `npx` CLI tool to `hardhat`:

BASH

```
$ npx hardhat compile
```

The `hardhat` tool automatically selects Solidity version `0.8.17` based on the version specified within the `hardhat.config.ts` file.

This version does not reproduce the expected bytecode of the live deployment of the system, and we proceeded to utilize the correct version `0.8.6`.

We advise `pragma` statements to be locked to `0.8.6` (`=0.8.6`), the same version utilized for our static analysis as well as optimizational review of the codebase.

During compilation with the `hardhat` pipeline, no errors were identified that relate to the syntax or bytecode size of the contracts.

The compiler of the contracts is misconfigured with an enabled optimizer that is ignored, and we advise the Term Finance team to rectify this discrepancy to achieve significant gas savings across their contracts.

CHAPTER 03

Static Analysis

Briefly describes the output of the static analyzers of the project.

TOTAL DETECTIONS	FALSE POSITIVES	ACCEPTED	COMPILER ERRORS
16	9	7	0

The execution of our static analysis toolkit identified **16 potential issues** within the codebase of which **9 were ruled out to be false positives** or negligible findings.

The remaining **7 issues** were validated and grouped and formalized into the **4 exhibits** that follow:

ConfigLockGuard.sol			CLG-S →
CLG-01S	Inexistent Sanitization of Input Address	INFO	ALLEV.
CLG-02S	Multiple Top-Level Declarations	INFO	ALLEV.

SetTxNonceGuard.sol			STN-S →
STN-01S	Inexistent Sanitization of Input Address	INFO	ALLEV.
STN-02S	Multiple Top-Level Declarations	INFO	ALLEV.

STATIC ANALYSIS CLG-S

ConfigLockGuard Static Analysis Findings

Contains all the findings that relate to static analysis on the contract codebase.

packages/evm/contracts/helpers/ConfigLockGuard.sol

CLG-01S INFO Inexistent Sanitization of Input Address

TYPE	DISPOSITION	LOCATION
Input Sanitization	• Alleviated	ConfigLockGuard.sol:L75-L77

DESCRIPTION

The linked function accepts an `address` argument yet does not properly sanitize it.

IMPACT

The presence of zero-value addresses, especially in `constructor` implementations, can cause the contract to be permanently inoperable. These checks are advised as zero-value inputs are a common side-effect of off-chain software related bugs.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L75-L77

```
75  constructor(address _lockedModifier) {
76      lockedModifier = _lockedModifier;
77  }
```

RECOMMENDATION

We advise some basic sanitization to be put in place by ensuring that the `address` specified is non-zero.

ALLEVIATED d29979f747

The input `_lockedDelay` address argument of the `ConfigLockGuard::constructor` function is adequately sanitized as non-zero in the latest in-scope revision of the codebase, addressing this exhibit.

CLG-02S INFO

Multiple Top-Level Declarations

TYPE	DISPOSITION	LOCATIONS
Code Style	• Alleviated	I-1 ConfigLockGuard.sol: L6 I-2 ConfigLockGuard.sol: L16 I-3 ConfigLockGuard.sol: L46

DESCRIPTION

The referenced file contains multiple top-level declarations that decrease the legibility of the codebase.

IMPACT

6|46|6,16,46

AFFECTED CODE 3 instances — excerpt shown

packages/evm/contracts/helpers/ConfigLockGuard.sol L6-L16

```
6  interface ISafeAdmin {
7      function setGuard(address guard) external;
8
9      function enableModule(address module) external;
10
11     function disableModule(address prevModule, address module) external;
12
13     function setFallbackHandler(address handler) external;
14 }
15
16 interface IModifierAdmin {
```

RECOMMENDATION

We advise all highlighted top-level declarations to be split into their respective code files, avoiding unnecessary imports as well as increasing the legibility of the codebase.

ALLEVIATED d29979f747

The relevant interfaces were relocated to dedicated files that are imported, addressing this exhibit.

STATIC ANALYSIS STN-S

SetTxNonceGuard Static Analysis Findings

Contains all the findings that relate to static analysis on the contract codebase.

`packages/evm/contracts/helpers/SetTxNonceGuard.sol`

STN-01S INFO Inexistent Sanitization of Input Address

TYPE	DISPOSITION	LOCATION
Input Sanitization	• Alleviated	SetTxNonceGuard.sol:L31-L33

DESCRIPTION

The linked function accepts an `address` argument yet does not properly sanitize it.

IMPACT

The presence of zero-value addresses, especially in `constructor` implementations, can cause the contract to be permanently inoperable. These checks are advised as zero-value inputs are a common side-effect of off-chain software related bugs.

AFFECTED CODE 1 instance

`packages/evm/contracts/helpers/SetTxNonceGuard.sol` L31-L33

```
31  constructor(address _delay) {
32      delay = _delay;
33  }
```

RECOMMENDATION

We advise some basic sanitization to be put in place by ensuring that the `address` specified is non-zero.

ALLEVIATED d29979f747

The input `_delay` address argument of the `SetTxNonceGuard::constructor` function is adequately sanitized as non-zero in the latest in-scope revision of the codebase, addressing this exhibit.

STN-02S INFO

Multiple Top-Level Declarations

TYPE	DISPOSITION	LOCATIONS
Code Style	• Alleviated	I-1 SetTxNonceGuard.sol: L6 I-2 SetTxNonceGuard.sol: L15

DESCRIPTION

The referenced file contains multiple top-level declarations that decrease the legibility of the codebase.

IMPACT

6|15|6,15

AFFECTED CODE

2 instances — excerpt shown

packages/evm/contracts/helpers/SetTxNonceGuard.sol L6-L15

```
6  interface IDelay {
7      function setTxNonce(uint256 _nonce) external;
8  }
9
10 /// @title SetTxNonceGuard - Restricts a module to calling setTxNonce on one
11 /// @dev Attach with `setGuard` on a Zodiac module. Module.exec and
12 /// Module.execAndReturnData route every transaction through checkTransactio
13 /// so while this guard is set the module can do nothing other than invalida
14 /// the queue of the Delay it was deployed against.
15 contract SetTxNonceGuard is BaseGuard {
```

RECOMMENDATION

We advise all highlighted top-level declarations to be split into their respective code files, avoiding unnecessary imports as well as increasing the legibility of the codebase.

ALLEVIATED d29979f747

The `IDelay` interface has been relocated to its dedicated file and is now imported properly, optimizing the code's style.

CHAPTER 04

Manual Review

Briefly describes the output of the manual review of the project.

A **thorough line-by-line review** was conducted on the codebase to identify potential malfunctions and vulnerabilities in Term Finance's custom Safe guards.

As the project at hand implements Gnosis Safe integrating infrastructure, intricate care was put into ensuring that the **security of the system conforms to the specifications and restrictions** laid forth within the integrated protocol's specification as well as their own.

We validated that **all state transitions of the system occur within sane criteria** and that all rudimentary formulas within the system execute as expected. We **pinpointed multiple deployment-related vulnerabilities** within the system which could have had **severe ramifications** to its overall operation, however, all are based on administrative misuse and are thus considered conditional on how the Term Finance team secures their multi-signature wallets.

Additionally, the system was investigated for any other commonly present attack vectors such as re-entrancy attacks, mathematical truncations, logical flaws and ERC / EIP standard inconsistencies. The documentation of the project was satisfactory to the extent it need be.

A total of **11 findings** were identified over the course of the manual review of which **8 findings** concerned the behaviour and security of the system. The non-security related findings, such as optimizations, are included in the separate Code Style chapter.

The finding table below enumerates all these security / behavioural findings:

ConfigLockGuard.sol			CLG-M →
CLG-01M	Misleading Generic Interface of Modifier Locking	INFO	ALLEV.
CLG-02M	Disassociation of Formal Verification & Deployed Contracts	MINOR	ALLEV.
CLG-03M	Setup-Based Bypass of Operation Restrictions	MINOR	ALLEV.
CLG-04M	Insufficient Restriction of Setters Resulting in Delay Bypass	MEDIUM	ALLEV.

PauseGuard.sol

PGD-M →

PGD-01M	ABI Incompatibility of Standard Pausable	INFO	NULLIFIED
PGD-02M	Breach of Expirability Trait	MINOR	ALLEV.
PGD-03M	Multi-Signature Threshold Power Discrepancy	MINOR	ACK

TermToken.sol

TTN-M →

TTN-01M	Privilege Escalation of Multisignature Wallet Power	MINOR	ALLEV.

MANUAL REVIEW CLG-M

ConfigLockGuard Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

packages/evm/contracts/helpers/ConfigLockGuard.sol

CLG-01M

INFO

Misleading Generic Interface of Modifier Locking

TYPE	DISPOSITION	LOCATION
Standard Conformity	• Alleviated	ConfigLockGuard.sol:L119-L141

DESCRIPTION

The `ConfigLockGuard::checkTransaction` function uses an `IModifierAdmin` generic interface to restrict calls to the underlying modifier, however, the contract itself does not actually lock all types of modifiers (f.e., a Roles V1 modifier can result in arbitrary delegate calls).

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L119-L141

```
119  } else if (to == lockedModifier) {
120      if (
121          selector == IModifierAdmin.transferOwnership.selector ||
122          selector == IModifierAdmin.renounceOwnership.selector
123      ) {
124          revert ModifierOwnershipLocked();
125      }
126      if (
127          selector == IModifierAdmin.enableModule.selector ||
128          selector == IModifierAdmin.disableModule.selector
129      ) {
130          revert ModifierModulesLocked();
```

```
131     }
132     if (selector == IModifierAdmin.setGuard.selector) {
133         revert ModifierGuardLocked();
134     }
135     if (
136         selector == IModifierAdmin.setAvatar.selector ||
137         selector == IModifierAdmin.setTarget.selector
138     ) {
139         revert ModifierAvatarLocked();
140     }
141 }
```

RECOMMENDATION

We advise the code to properly use a strict `IDelay` interface to showcase that it is built for the Delay modifier specifically.

● ALLEVIATED d29979f747

The code was revised to use an appropriate `IDelay` interface, addressing this exhibit.

CLG-02M

MINOR

Disassociation of Formal Verification & Deployed Contracts

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	ConfigLockGuard.sol:L47-L49, L75-L77

DESCRIPTION

The formal verification rules under `packages/evm/certora/specs/Safe/ownerlessSafeGuard.spec` state that the `OwnerlessSafe` is built with `lockedModifier = address(0)`, however, this assumption is breached on the actual deployment of the system as all contracts share the same `ConfigLockGuard` instance pointing to the `Delay` contract.

IMPACT

The formal verification rules the project is meant to uphold are breached by its deployment, albeit in an non-impactful way.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L47-L49

```
47  /// @dev Zodiac modifier whose ownership, modules, guard, avatar and target
48  /// are locked.
49  address public immutable lockedModifier;
```

RECOMMENDATION

We advise the formal verification rules to be updated, either dropping the requirement altogether or pinning it to the Delay address.

• ALLEVIATED d29979f747

The formal verification rules were updated to reflect this fact, alleviating this exhibit.

CLG-03M

MINOR

Setup-Based Bypass of Operation Restrictions

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	ConfigLockGuard.sol:L32

DESCRIPTION

The `ConfigLockGuard` and surrounding infrastructure (TERM governance, etc.) are meant to restrict the speed at which the TERM system contracts are upgraded or reconfigured.

However, the actual deployment of the system relies on an `OwnerlessSafe` which has 9 active EOA signers and is setup on a 9-out-of-9 setup.

Should these 9 actors collude, it is possible to arbitrarily upgrade the contracts and take over the system bypassing all veto power and pause safeguards.

IMPACT

The setup of the system permits a 9-out-of-9 Safe to rewrite its threshold (i.e. converting it to a 1-out-of-1) and upgrade system contracts without flowing through the various guards of the system.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L32

```
32  /// @title ConfigLockGuard - Locks a Safe's configuration, and that of a Zod
```

RECOMMENDATION

We advise this trait to be properly documented in the system's documentation, or the `OwnerlessSafe` to be updated to contain a module that prevents transaction executions.

ALLEVIATED d29979f747

This trait has been properly documented across the system's proofs and the 9-out-of-9 direct path is considered a trust assumption.

We would like to note the top-level `plan` documents do not reflect this change, however, we consider it addressed as the proofs act as the source-of-truth for how the system behaves.

CLG-04M

MEDIUM

Insufficient Restriction of Setters Resulting in Delay Bypass

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	ConfigLockGuard.sol:L43 -L45, L119-L141

DESCRIPTION

The `ConfigLockGuard` documentation states that the `Delay`-related functions such as `setTxNonce`, `setTxCooldown`, and `setTxExpiration` deliberately remain open.

By allowing these functions to be invoked, the system permits a bypass of its intended delay-based security mechanisms by proposing and executing a proposal in the same block / transaction.

In detail, the master copy `Delay` that has been deployed has a `DelayOwnerSafe` configured as its owner and a `ProposerSafe` as its only module. Both of these safes have the same 11 owners and the same 5-out-of-11 threshold.

As a result, 5 colluding users can create a transaction that executes `Delay::setTxCooldown(0)` and run `executeNextTx` right afterwards, bypassing the normal delay.

The normal transaction delay is security-sensitive, as the documented flow for TERM holders to veto abuse of power is through their `Governor` which has a `votingDelay` of `0` and a `votingPeriod` of `79200`. Any transaction cooldown less than `79200` nullifies the TERM veto pathway.

IMPACT

In contrast to the documentation of the system, the 5-out-of-11 signer threshold can upgrade all system contracts in the same block without any window for TERM holders to react and veto, effectively defeating the security purpose the TERM's `Governor` is meant to fulfill.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L119-L141

```
119  } else if (to == lockedModifier) {
120      if (
121          selector == IModifierAdmin.transferOwnership.selector ||
```

```
122             selector == IModifierAdmin.renounceOwnership.selector
123         ) {
124             revert ModifierOwnershipLocked();
125         }
126         if (
127             selector == IModifierAdmin.enableModule.selector ||
128             selector == IModifierAdmin.disableModule.selector
129         ) {
130             revert ModifierModulesLocked();
131         }
132         if (selector == IModifierAdmin.setGuard.selector) {
133             revert ModifierGuardLocked();
134         }
135         if (
136             selector == IModifierAdmin.setAvatar.selector ||
137             selector == IModifierAdmin.setTarget.selector
138         ) {
139             revert ModifierAvatarLocked();
140         }
141     }
```

RECOMMENDATION

We advise the `lockedModifier` logic branch to enforce an allowlist instead, decoding the expected input arguments of calls such as `setTxCooldown` and `setTxExpiration` to restricted bounds (f.e. the `Governor::votingPeriod` plus a buffer).

Alternatively, reject those transaction selectors outright.

● ALLEVIATED d29979f747

The code was updated to prevent `IDelay::setTxCooldown` or `IDelay::setTxExpiration` calls; while the live mainnet remains susceptible to this issue, a new live proof has been provided to make this explicit by holding `txCooldown` constant and outlining that `DelayOwnerSafe` can change the `txCooldown` and `txExpiration`.

As a result, we consider this exhibit alleviated to the best extent possible by the Term Finance team.

MANUAL REVIEW PGD-M

PauseGuard Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

`packages/evm/contracts/helpers/PauseGuard.sol`

PGD-01M INFO ABI Incompatibility of Standard Pausable

TYPE	DISPOSITION	LOCATIONS
Standard Conformity	• Nullified	I-1 PauseGuard.sol:L26 I-2 PauseGuard.sol:L27

DESCRIPTION

The `PauseGuard` appears to have the same event signatures as OpenZeppelin's `Pausable` implementation yet uses the `indexed` keyword on its arguments.

AFFECTED CODE 2 instances — excerpt shown

`packages/evm/contracts/helpers/PauseGuard.sol` L26-L27

```
26  event Paused(address indexed account);
27  event Unpaused(address indexed account);
```

RECOMMENDATION

We advise the `indexed` keyword to be omitted if compatibility with listeners built for OpenZeppelin's `Pausable` is expected.

• NULLIFIED d29979f747

The Term Finance team evaluated this exhibit and clarified that compatibility with OpenZeppelin `Pausable` listeners is not expected, rendering the exhibit nullified.

PGD-02M MINOR

Breach of Expirability Trait

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	PauseGuard.sol:L58-L60

DESCRIPTION

Per the `PauseGuard::pause` documentation and the specification of the system, a transaction that expires while paused is meant to become inexecutable.

This trait can be breached via a re-configuration of the transaction expiration or cooldown (functions that are deliberately accessible per `ConfigLockGuard`), thereby allowing an expired queued entry to become executable after it has expired.

IMPACT

The documentation of the system relies on a security trait around queue expiry that is not guaranteed.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/PauseGuard.sol L58-L70

```
58  /// @dev Blocks every execution through the Delay until an ADMIN_ROLE holder
59  /// unpauses. Entries already in the queue keep ageing while paused, so any
60  /// entry whose expiration passes during the pause can never execute.
61  function pause() external {
62      if (msg.sender ≠ pauser) {
63          revert NotPauser(msg.sender);
64      }
65      if (paused) {
66          revert AlreadyPaused();
67      }
68      paused = true;
69      emit Paused(msg.sender);
70  }
```

RECOMMENDATION

Correct the documentation of the system and update the operational runbook of the codebase to ensure that `skipExpired` is invoked whenever a transaction expires to prevent it from being revived.

RESPONSE d29979f747

The NatSpec was corrected to outline the potential for an expired transaction to be revived, and outlines the `Delay::skipExpired` step, however, the runbooks of the system have not been updated to reflect this.

We advise the runbooks to be updated to include the `Delay::skipExpired` step to ensure that the issue does not manifest itself in practice.

ALLEVIATED 69e22d825b

The Term Finance team updated its private operational runboooks (evaluated at commit `9a0c551cf2fc0c37a58043b43a8f1a5b068f7339`) to illustrate how the `Delay::skipExpired` function must be used to ensure the system behaves securely, alleviating this exhibit in full.

PGD-03M

MINOR

Multi-Signature Threshold Power Discrepancy

TYPE	DISPOSITION	LOCATION
Logical Fault	• Acknowledged	PauseGuard.sol:L58-L70, L72-L80, L85-L91

DESCRIPTION

The overall voting flow of the Term Finance system is meant to allow veto powers for a 5-out-of-11 multi-signature wallet or a TERM governance vote.

In the current way the deployment `PauseGuard` is setup, a 4-out-of-10 wallet that holds the `ADMIN_ROLE` in the `PauseGuard` can reset the contract's pauser and pause the system indefinitely, effectively acting as veto power for transactions given that they will eventually expire.

IMPACT

A 4-out-of-10 signature can supercede a 5-out-of-11 signature and prevent a transaction from being executed.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/PauseGuard.sol L58-L91

```
58  /// @dev Blocks every execution through the Delay until an ADMIN_ROLE holder
59  /// unpauses. Entries already in the queue keep ageing while paused, so any
60  /// entry whose expiration passes during the pause can never execute.
61  function pause() external {
62      if (msg.sender ≠ pauser) {
63          revert NotPauser(msg.sender);
64      }
65      if (paused) {
66          revert AlreadyPaused();
67      }
68      paused = true;
69      emit Paused(msg.sender);
70  }
71
72  /// @dev Lifts the pause. Every queue entry that is past its cooldown and
73  /// has not expired becomes executable by anyone again immediately.
74  function unpause() external onlyRole(ADMIN_ROLE) {
```

```
75      if (!paused) {
76          revert NotPaused();
77      }
78      paused = false;
79      emit Unpaused(msg.sender);
80  }
81
82  /// @dev Replaces the account allowed to pause. There is no way to leave it
83  /// unset: to stand the pauser down, point it at an account that cannot act.
84  /// @param account New pauser
85  function setPauser(address account) external onlyRole(ADMIN_ROLE) {
86      if (account == address(0)) {
87          revert ZeroAddress();
88      }
89      emit PauserSet(pauser, account);
90      pauser = account;
91  }
```

RECOMMENDATION

We advise the `ADMIN_ROLE` to be a role that either adheres to the same threshold restrictions as the 5-out-of-11 multi-signature wallet interacting with the `Delay` contract, and allow the role to also be handled by the governance of the TERM token to permit a community recovery path.

● RESPONSE d29979f747

The Term Finance team stated that future deployments will assign the `PauseGuard` administrator to the 5-out-of-11 `ProposerSafe` (as illustrated in the updated formal verification proofs), while the live deployment remains unchanged.

No governance recovery path has been introduced, rendering this exhibit partially addressed.

ACKNOWLEDGED 69e22d825b

The Term Finance team updated its private operational runboooks (evaluated at commit `9a0c551cf2fc0c37a58043b43a8f1a5b068f7339`) to accommodate for pauser rotations and pause state changes, however, no governance recovery path has been introduced.

The Term Finance team reviewed this recommendation, and opted to acknowledge it as they consider the system to behave per their business requirements, rendering this exhibit acknowledged.

MANUAL REVIEW TTN-M

TermToken Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

packages/evm/certora/vendor/TermToken/src/TermToken.sol

TTN-01M

MINOR

Privilege Escalation of Multisignature Wallet Power

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	TermToken.sol:L225-L227

DESCRIPTION

The `TermToken` that the governor with veto power depends on is a mintable token whose mint capabilities are guarded by the `PAUSER_ROLE`.

In the live deployment of the system, this role is held by a 4-out-of-10 multisignature wallet thereby permitting the token to be minted arbitrarily via a significantly lower threshold than other operations of the system.

IMPACT

The capability to mint the TERM token is restricted by a 4-out-of-10 multisignature wallet instead of the governor or the 5-out-of-11 canonical multisignature wallet of Term Finance.

AFFECTED CODE 1 instance

packages/evm/certora/vendor/TermToken/src/TermToken.sol L225-L227

```
225 function unpauseMinting() external onlyRole(PAUSER_ROLE) {
226     mintingNotPaused = true;
227 }
```

RECOMMENDATION

We advise minting to either become frozen permanently or the `PAUSER_ROLE` to be transitioned to a different party, ensuring that mint capabilities are restrained by the

highest security restrictions available.

RESPONSE d29979f747

The Term Finance team prepared an upgraded `TermToken` implementation in a separate repository that restricts `TermToken::mint` and `TermToken::burn` to the `DEFAULT_ADMIN_ROLE` and removes the LayerZero mint path, and stated that the `PAUSER_ROLE` will be moved from the 4-out-of-10 `AdminSafe` to the `ProposerSafe`. A copy of this file exists at `packages/evm/certora/vendor/TermToken/src-triage/TermToken.sol`, whose behaviour we validated on a main-net fork.

However, the in-scope `TermToken` file remains unchanged in the revision of the codebase we have covered on this iteration round, the TERM proxy has not been upgraded, and the `PAUSER_ROLE` remains with the `AdminSafe`, rendering this exhibit open.

ALLEVIATED 69e22d825b

The Term Finance team upgraded the TERM proxy to a new `TermToken` implementation (`0x839B00C12b3892764c7ca1f73896325b34de9204`) whose source is available in the public `term-finance/term-token-contracts` repository at commit `fa1722faf0bc0e953efc65394e35e7919cb2793d` as shown here.

The `TermToken::mint` and `TermToken::burn` functions are now restricted to the `DEFAULT_ADMIN_ROLE` in addition to their pause flags, and the LayerZero `OFTCore` credit path has been removed, ensuring that the token's supply can solely be increased by the `DEFAULT_ADMIN_ROLE` held by the `OwnerlessSafe`.

We confirmed that the deployed runtime bytecode matches the audited code apart from its metadata hash, and that the proxy's storage and state were preserved, alleviating this exhibit in full.

CHAPTER 05

Code Style

Describes the dedicated section of coding style and optimizational findings of the audit

During the manual portion of the audit, we identified **3 optimizations** that can be applied to the codebase that will decrease the operational cost associated with the execution of a particular function and generally ensure that the project complies with the latest best practices and standards in Solidity.

Additionally, this section of the audit contains any opinionated adjustments we believe the code should make to make it more legible as well as truer to its purpose.

These optimizations are enumerated below:

ConfigLockGuard.sol			CLG-C →
CLG-01C	Overstatement of Recoverability	INFO	ALLEV.
CLG-02C	Preemptive Extraction of Selector	INFO	ALLEV.

PauseGuard.sol			PGD-C →
PGD-01C	Overcomplicated Role-Based Access Control	INFO	ALLEV.

CODE STYLE CLG-C

ConfigLockGuard Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

packages/evm/contracts/helpers/ConfigLockGuard.sol

CLG-01C INFO Overstatement of Recoverability

TYPE	DISPOSITION	LOCATION
Standard Conformity	• Alleviated	ConfigLockGuard.sol:L40-L41

DESCRIPTION

The `ConfigLockGuard` documentation states that:

> The Safe is taken from msg.sender rather than pinned, so the guard never rejects a Safe it was installed on by mistake with no way to take it off again.

This statement is incorrect, as a Safe that installs the `ConfigLockGuard` with no other module installed will become bricked.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L39-L41

```
39  /// set, the guard cannot be removed or replaced. The Safe is taken from
40  /// msg.sender rather than pinned, so the guard never rejects a Safe it was
41  /// installed on by mistake with no way to take it off again.
```

RECOMMENDATION

We advise the documentation to be corrected, properly illustrating that accidentally enabling the guard on a new safe can result in an irrecoverable state.

ALLEVIATED d29979f747

The NatSpec has been updated to reflect that the guard is permanent on a Safe without a module.

CLG-02C INFO

Preemptive Extraction of Selector

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	ConfigLockGuard.sol:L104

DESCRIPTION

The `bytes4` selector of the input `data` is extracted before the address-specific conditionals that trigger its comparison, rendering its extraction to be wasteful whenever the `to` address does not match the restricted targets.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/ConfigLockGuard.sol L104-L141

```
104  bytes4 selector = bytes4(data);
105
106  if (to == msg.sender) {
107      if (selector == ISafeAdmin.setGuard.selector) {
108          revert GuardLocked();
109      }
110      if (
111          selector == ISafeAdmin.enableModule.selector ||
112          selector == ISafeAdmin.disableModule.selector
113      ) {
114          revert ModulesLocked();
115      }
116      if (selector == ISafeAdmin.setFallbackHandler.selector) {
117          revert FallbackHandlerLocked();
118      }
119  } else if (to == lockedModifier) {
120      if (
121          selector == IModifierAdmin.transferOwnership.selector ||
122          selector == IModifierAdmin.renounceOwnership.selector
123      ) {
124          revert ModifierOwnershipLocked();
125      }
126      if (
127          selector == IModifierAdmin.enableModule.selector ||
128          selector == IModifierAdmin.disableModule.selector
129      ) {
130          revert ModifierModulesLocked();
```

```
131     }
132     if (selector == IModifierAdmin.setGuard.selector) {
133         revert ModifierGuardLocked();
134     }
135     if (
136         selector == IModifierAdmin.setAvatar.selector ||
137         selector == IModifierAdmin.setTarget.selector
138     ) {
139         revert ModifierAvatarLocked();
140     }
141 }
```

RECOMMENDATION

We advise the selector extraction to be relocated within the `if` blocks, optimizing the code's gas cost.

● ALLEVIATED d29979f747

The selector is now extracted within each `if-else-if` branch, optimizing the code's gas cost.

CODE STYLE PGD-C

PauseGuard Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

packages/evm/contracts/helpers/PauseGuard.sol

PGD-01C INFO Overcomplicated Role-Based Access Control

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	PauseGuard.sol:L15

DESCRIPTION

The `PauseGuard` contract inherits `AccessControl` whilst disabling its role-assignment control surface and retaining a single role and address association in the contract's lifetime.

AFFECTED CODE 1 instance

packages/evm/contracts/helpers/PauseGuard.sol L15

```
15  contract PauseGuard is AccessControl, IGuard {
```

RECOMMENDATION

We advise the code to opt for a simplified single-address role assignment, greatly optimizing the code's gas cost.

• ALLEVIATED d29979f747

The code no longer inherits `AccessControl` and instead relies on a single immutable `admin` variable, optimizing the code's gas cost significantly.

CHAPTER 06

Deployment Validation of Gnosis Safe Infrastructure

A dedicated chapter that outlines the deployment validation steps our team undertook to cross-reference the documents provided to us by the Term Finance team with the live blockchain state.

The Term Finance team requested that we validate the deployed bytecode of the smart contracts located at `0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`, `0x7aE03372ECDcEe335CdEfB7d354d507F0506616C`, and `0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53` on the Ethereum mainnet matches the source code that was in scope for this security audit.

We were able to re-generate the precise byte-for-byte deployments of the audited `PauseGuard`, `SetTxNonceGuard`, and `ConfigLockGuard` contracts and specifically:

- Commit Hash: b78b57ea6e37c3eee48e265acb3c3dca4f76d5ff
- Scope: `packages/evm/contracts/helpers/PauseGuard.sol`, `packages/evm/contracts/helpers/SetTxNonceGuard.sol`, `packages/evm/contracts/helpers/ConfigLockGuard.sol`

As part of this review, we also assessed the potential impact of the findings identified in this audit report to the live deployment of the contracts.

While only the three aforementioned contracts were audited, we additionally validated the live deployed state of all other contracts comprising the Term Finance governance system per the documentation provided by the Term Finance team.

Bytecode Equivalence

To confirm that the deployed bytecode matched the local version of the codebase we audited, we traced the compilation configuration utilized to the following parameters:

Parameter	Value
Compiler	`solc 0.8.6+commit.11564f7e`
Optimizer	**Disabled**
Optimizer Runs	200

Parameter	Value
EVM Version	Berlin
Bytecode Hash (`PauseGuard`)	IPFS (`QmXsvfgXu5wC8NK5SAvC2YQDiFbG4BMdyqEYqLm1juNWpA`)
Bytecode Hash (`SetTxNonceGuard`)	IPFS (`QmZ1NhtmgUHquMLZYinv52txKc9XkKdT8UyiwC9sBcEX2A`)
Bytecode Hash (`ConfigLockGuard`)	IPFS (`QmRb5jzWeXhTYex1XZhYGeEWZZfmC6KESqQMYhgyd9wrQE`)
Use Literal Content Flag	`true`
Zodiac Dependency Version	`1.0.1`
Safe Dependency Version	`1.3.0`
OpenZeppelin Dependency Version	`4.9.6` (`PauseGuard` only)

The compiler version configured in the repository's root `hardhat.config.ts` file should be disregarded as it does not correspond to the deployment and would result in different bytecode generations. Additionally, the optimizer configuration declared in the `packages/evm/hardhat.config.ts` file (`enabled: true`, `runs: 1`) is silently ignored by Hardhat as it is specified outside of the per-compiler entries, resulting in the optimizer being disabled with its default runs value.

While the optimizer was disabled for producing the bytecode of the guards, the runs value of `200` is required to ensure the same IPFS hashes are generated and, in the case of the `PauseGuard`, that the actual bytecode matches due to how the compiler relies on this value to set its function-matching bytecode irrespective of whether the optimizer is enabled.

We were able to re-generate the full deployment bytecode of all three contracts and confirm via multiple JSON-RPC providers that it matches the deployed code.

Live Deployment State

The contracts represent immutable deployments that cannot be upgraded and contain no instructions that permit arbitrary code execution (i.e. `DELEGATECALL` etc.).

The contracts were deployed as follows:

Data Point	`PauseGuard`	`SetTxNonceGuard`	`ConfigLockGuard`
Creation Transaction	`0xcbe1ae8cdf1935188bc190156a65062c061e98026426b4cc71fb3d0846e37ab9`	`0x919d6b1ea50e4a763c71e83c204b02f23170e361b395afd36a3e3277b3634e83`	`0x580d120cf7798a8aa2962bdf8cb4026e72d6271bad2848b96cceb090b1aaef83`
Block	26,041,840 (`0xf388b78b4f43d6a95ea04650a4ee306a3f4d5ac9277480a4dc736c724139afc8`), 2026-09-23 18:03:47 UTC	26,041,911 (`0x6dd412d0a653edf87942a6de194671f90fcd484efa2ce4094351a97bc67758f0`), 2026-09-23 18:18:23 UTC	26,086,454 (`0x136582b355939807b6c61800bccc2627725e775916e9385ae1b9da138edac692`), 2026-09-29 23:36:59 UTC
Deployer	`0xDaCE6985E42EC10F492D0919493964922b833b5B` , nonce 3497	`0xDaCE6985E42EC10F492D0919493964922b833b5B` , nonce 3498	`0xDaCE6985E42EC10F492D0919493964922b833b5B` , nonce 3509

The on-chain state changes confirm that the `ADMIN_ROLE` of the `PauseGuard` was granted to the `0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774` address and that its pauser was set to the `0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472` address.

In detail, the following constructor arguments were used:

Argument	Value
`PauseGuard::_admin`	0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774
`PauseGuard::_pauser`	0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472
`SetTxNonceGuard::_delay`	0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf
`ConfigLockGuard::_lockedModifier`	0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf

The constructor arguments match the repository's deployment plan. We observed a discrepancy in the repository's governance plan (`new_governance_plan_ownerless_safe.md`), which still documents two dedicated guards (`ProposerSafeGuard` at `0xd9fA1D4ee16CBc1178609B50150ab562bac737DD` and `DelayOwnerGuard` at `0x6Ede0489a7A4Eb88c2bC487aeeaD20303E155bDa`) that were deployed by the same deployer but whose source code is not part of the audited commit; both were superseded by the `ConfigLockGuard` and are not installed anywhere in the deployment. We advise the Term Finance team to rectify this for future verification attempts.

To assess the latest state of the guard contracts, we queried their state at block `26_182_517` (2026-10-13 08:57:23 UTC):

Live Configuration	Value
`PauseGuard::paused()`	`false`
`PauseGuard::pauser()`	0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472
`PauseGuard::hasRole(ADMIN_ROLE, …)`	0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774 (sole holder)
`PauseGuard::getRoleAdmin(ADMIN_ROLE)`	0x00…00 (`DEFAULT_ADMIN_ROLE` , no holder)
`SetTxNonceGuard::delay()`	0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf
`ConfigLockGuard::lockedModifier()`	0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf
`Delay::getGuard()`	0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618 (`PauseGuard`)
`NewRoles::guard()`	0x7aE03372ECDcEe335CdEfB7d354d507F0506616C (`SetTxNonceGuard`)
Guard of the OwnerlessSafe, ProposerSafe, and DelayOwnerSafe	0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53 (`ConfigLockGuard`)

The `ADMIN_ROLE` of the `PauseGuard` , which is capable of unpausing executions and replacing the pauser, has been permanently assigned to a Safe proxy of version `1.3.0` with a threshold of **4-of-10**, while its pauser points to a Safe proxy of version `1.4.1` with a threshold of **2-of-9**. The `ConfigLockGuard` is shared by three Safe proxies with thresholds of **9-of-9** (`1.3.0`), **5-of-11** (`1.3.0`), and **5-of-11** (`1.4.1`), the last of which owns the Delay modifier referenced by both the `ConfigLockGuard` and the `SetTxNonceGuard` , while the NewRoles modifier guarded by the `SetTxNonceGuard` has had its ownership renounced.

The Delay modifier guarded by the `PauseGuard` enforces a cooldown and an expiration of 86.400 seconds (24 hours) each, and its queue was empty at the time of our query (`txNonce` and `queueNonce` equal to `208`).

Beyond the audited guards, we validated the live deployed state of every other contract comprising the governance system at the same block against the documentation provided by the Term Finance team (`GovernanceConfigurationProof.pdf` , `Term Gov Mainnet Addresses.pdf` , `term_governance_contracts_live_state.pdf` , and `term_governance_map.pdf`). Each file is associated with the following SHA-256 digests:

File	Size (bytes)	SHA-256
GovernanceConfigurationProof.pdf	4,814,298	cbe932b3b89d82eaf02201a4c7d4529211011fc46c18e418ba0be36e910f3c49
Term Gov Mainnet Addresses.pdf	292,319	2d770a208c33e53d449c37764f3f8aef44f521c6509c612388cf183b4fd23ede
term_governance_contracts_live_state.pdf	296,924	c584aadfb14aca7b867139cf50154eb86b14f50f170d38d4495c72d0da38f5f6
term_governance_map.pdf	232,972	fb6ce7c1279f200f8be0f989d4dc2a39c533aeb952222f8e71df6600b5b9cf1c

The contracts validated are the following:

Contract	Address	Implementation
OwnerlessSafe	0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03	Safe `1.3.0` (`0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`)
ProposerSafe	0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28	Safe `1.3.0` (`0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`)
DelayOwnerSafe	0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3	Safe `1.4.1` (`0x41675C099F32341bf84BFc5382aF534df5C7461a`)
PauseSafe	0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472	Safe `1.4.1` (`0x41675C099F32341bf84BFc5382aF534df5C7461a`)
AdminSafe	0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774	Safe `1.3.0` (`0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`)
Delay	0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf	EIP-1167 clone of Delay `1.0.1` (`0xd54895B1121A2eE3f37b502F507631FA1331BED6`)
NewRoles	0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3	EIP-1167 clone of Roles `1.0.0` (`0x85388a8cd772b19a468F982Dc264C238856939C9`)
Old Roles	0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2	Roles v1 (disconnected)

Contract	Address	Implementation
Governor	0x2B715634134220ffeEE9458b4e34E41A41418607	`TermFinanceGovernor` (immutable)
TERM	0xC3d21f79C3120A4fFda7A535f8005a7c297799bF	ERC-1967 proxy of `TermToken` (`0xeC222d8AfB8b4E78C418ebc1ab2cA181f19FbadC`)
TermController	0xd902EBb8AEb832643af38f43d466a6155cD8Bd5A	ERC-1967 proxy
TermEventEmitter	0xf021B31282a60528B2F47D07Ce353da870be78b3	ERC-1967 proxy
TermPriceConsumerV3	0x13Ca4ddB295d621761057D682a1c7b5f5D7Bba4c	ERC-1967 proxy
TermRepoDeployerFactory	0x44058C32B154A516F3b9DE2413e6cb938F93c3B4	Immutable
TermInitializer	0x3B668F408293e9a5700D6Eeae78E1f945CF501eF	Immutable
TermDiamond	0xb4B1de03F229220eDbBB31A224b2a004b3c3e47f	EIP-2535 diamond

The Safe singletons are the canonical `1.3.0` and `1.4.1` deployments, and the modifier clones point to the canonical Zodiac mastercopies. The Safes were validated to be configured as follows:

Variable	OwnerlessSafe	ProposerSafe	DelayOwnerSafe	PauseSafe	AdminSafe
`getThreshold()` / owners	9-of-9	5-of-11	5-of-11	2-of-9	4-of-10
Guard	`ConfigLockGuard`	`ConfigLockGuard`	`ConfigLockGuard`	None	None
Fallback Handler	None	None	None	None	None
`getModulesPaginated()`	Delay	None	NewRoles	None	None
`nonce()`	16	169	4	1	785

Each Safe's owners were validated to be the following addresses, all of which are externally owned accounts:

Owner	OwnerlessSafe	ProposerSafe	DelayOwnerSafe	PauseSafe	AdminSafe
0xc1047a4D9f6071B55585523F7F659F39A1fBA74b	✓	✓	✓	✓	✓
0x8EF485fD38b7B29a827938C9F00300eaBf8E1710	✓	✓	✓		✓
0xbfFcAdCd5549cC378693108BcD4435776A6fa795	✓	✓	✓	✓	✓
0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3	✓	✓	✓	✓	✓
0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60	✓	✓	✓	✓	✓
0xE82183cfAE1C24f044315c318FD97bE7e47b31D2	✓	✓	✓		✓
0xeee661edcFE634Dc0e29D62C26AfD62c0843b817	✓	✓	✓	✓	✓
0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81	✓	✓	✓	✓	✓

Owner	OwnerlessSafe	ProposerSafe	DelayOwnerSafe	PauseSafe	AdminSafe
0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D	✓	✓	✓	✓	✓
0xF6A745f9B38FFcd17Ee8909AC89151D788F95282		✓	✓	✓	✓
0xC43d527E3544A3d199Ce28E87994384D89d90e6E		✓	✓		
0xC3CbFc5DA4B3d4B1258D63cA7ba56518C33f28c7				✓	

The Zodiac modifiers were validated to be configured as follows, with the role configuration of the `NewRoles` modifier read directly from its storage:

Variable	Delay	NewRoles
`owner()`	DelayOwnerSafe	None (renounced)
`avatar()`	OwnerlessSafe	OwnerlessSafe
`target()`	OwnerlessSafe	DelayOwnerSafe
Guard	`PauseGuard`	`SetTxNonceGuard`
`getModulesPaginated()`	ProposerSafe	Governor
`txCooldown()` / `txExpiration()`	86.400 / 86.400	-
`txNonce()` / `queueNonce()`	208 / 208	-

Variable	Delay	NewRoles
`multisend()`	-	0x9641d764fc13c8B624c04430C7356C1C7C8102e2 (MultiSendCallOnly `1.4.1`)
`defaultRoles(Governor)`	-	1
Role `1`	-	Governor as sole member; Delay as sole target, scoped at function level; only `setTxNonce(uint256)` permitted, without value or delegate calls and with unrestricted parameters

The Old Roles modifier has been fully disconnected from the system; its `owner()`, `avatar()`, and `target()` are all set to the zero address, and it is no longer a module of the OwnerlessSafe or of the Delay.

The Governor and the TERM token were validated to be configured as follows, with the role assignments of the token representing its sole role holders per its full event history:

Live Configuration	Value
`Governor::name()` / `version()`	"TermFinanceGovernor" / "1"
`Governor::token()`	0xC3d21f79C3120A4fFda7A535f8005a7c297799bF (TERM)
`Governor::votingDelay()`	0
`Governor::votingPeriod()`	79.200 (22 hours)
`Governor::proposalThreshold()`	1.000×10^{18}
`Governor::quorum()`	$1.000.000 \times 10^{18}$ (1% of the supply)
`Governor::COUNTING_MODE()`	"support=bravo&quorum=for,abstain"
`Governor::CLOCK_MODE()`	"mode=timestamp&from=default"
`Governor::timelock()`	None (not implemented)
`TERM::name()` / `symbol()` / `decimals()`	"Term Finance" / "TERM" / 18
`TERM::totalSupply()`	$100.000.000 \times 10^{18}$
`TERM` ERC-1967 implementation / admin	0xeC222d8AfB8b4E78C418ebc1ab2cA181f19FbadC / None
`TERM::owner()`	OwnerlessSafe

Live Configuration	Value
`TERM` `DEFAULT_ADMIN_ROLE` / `UPGRADER_ROLE`	OwnerlessSafe / OwnerlessSafe
`TERM` `PAUSER_ROLE`	AdminSafe
`TERM::paused()`	`false`
`TERM::mintingNotPaused()` / `burningNotPaused()`	`false` / `false`
`TERM::endpoint()` / LayerZero peers	0x1a44076050125825900e736c501f859c50fE728c / None

Finally, the `DEVOPS_ROLE` of each Term protocol contract was validated to be solely held by the OwnerlessSafe, with the `DEFAULT_ADMIN_ROLE` remaining unassigned:

Contract	`DEVOPS_ROLE` Holder	`DEVOPS_ROLE` Admin
TermController	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)
TermEventEmitter	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)
TermPriceConsumerV3	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)
TermRepoDeployerFactory	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)
TermInitializer	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)
TermDiamond	OwnerlessSafe	`DEFAULT_ADMIN_ROLE` (no holder)

All addresses and contract-level values stated in the documentation matched the live state, apart from the following documentation inaccuracies: the address list names the ConfigLockGuard as the Delay's guard (it is the PauseGuard), dates the renounced ownership of the NewRoles modifier to a block preceding the renouncement, orders some Safe owners differently than on-chain, and lists the disconnected Old Roles modifier among the core contracts. Additionally, the governance map still marks the DelayOwnerSafe's ConfigLockGuard as pending installation.

Applicability of Findings to Live State

Several findings identified within the audit report pertain directly to the deployment and setup of the various Safe implementations in use by the Term Finance team.

The main security concern with the guards in their current state is that the `ConfigLockGuard` leaves the timing setters of the Delay modifier open permitting a veto / pause bypass. Additionally, the permanent `ADMIN_ROLE` holder of the `PauseGuard` can freeze the Delay indefinitely, and the owners of the Safe the Delay executes on retain a direct execution path outside of it as all nine of their keys are live. Beyond the guards, the `PAUSER_ROLE` of the TERM token held by the same 4-of-10 Safe can render minting permissionless, allowing any party to acquire the voting power required to veto every queued transaction.

With the exception of the veto-path behaviours, which depend on how token holders formulate and time their proposals, all of these vulnerabilities can only be exploited via privilege misuse or key compromise of the Term Finance Safes; thus, as long as the Term Finance team responsibly maintains their Safes and adheres to the documented veto procedures, they are of no immediate concern, albeit they weaken the guarantees described in the project's governance documentation.

APPENDIX

Finding Types

A description of each finding type included in the report can be found below and is linked by each respective finding. A full list of finding types Omniscia has defined will be viewable at the central audit methodology we will publish soon.

External Call Validation	Many contracts that interact with DeFi contain a set of complex external call executions that need to happen in a particular sequence and whose execution is usually taken for granted whereby it is not always the case. External calls should always be validated, either in the form of `require` checks imposed at the contract-level or via more intricate mechanisms such as invoking an external getter-variable and ensuring that it has been properly updated.
Input Sanitization	As there are no inherent guarantees to the inputs a function accepts, a set of guards should always be in place to sanitize the values passed in to a particular function.
Indeterminate Code	These types of issues arise when a linked code segment may not behave as expected, either due to mistyped code, convoluted `if` blocks, overlapping functions / variable names and other ambiguous statements.

Language Specific	Language specific issues arise from certain peculiarities that the Solidity language boasts that discerns it from other conventional programming languages. For example, the EVM is a 256-bit machine meaning that operations on less-than-256-bit types are more costly for the EVM in terms of gas costs, meaning that loops utilizing a `uint8` variable because their limit will never exceed the 8-bit range actually cost more than redundantly using a `uint256` variable.
Code Style	An official Solidity style guide exists that is constantly under development and is adjusted on each new Solidity release, designating how the overall look and feel of a codebase should be. In these types of findings, we identify whether a project conforms to a particular naming convention and whether that convention is consistent within the codebase and legible. In case of inconsistencies, we point them out under this category. Additionally, variable shadowing falls under this category as well which is identified when a local-level variable contains the same name as a contract-level variable that is present in the inheritance chain of the local execution level's context.
Gas Optimization	Gas optimization findings relate to ways the codebase can be optimized to reduce the gas cost involved with interacting with it to various degrees. These types of findings are completely optional and are pointed out for the benefit of the project's developers.
Standard Conformity	These types of findings relate to incompatibility between a particular standard's implementation and the project's implementation, oftentimes causing significant issues in the usability of the contracts.

Mathematical Operations	In Solidity, math generally behaves differently than other programming languages due to the constraints of the EVM. A prime example of this difference is the truncation of values during a division which in turn leads to loss of precision and can cause systems to behave incorrectly when dealing with percentages and proportion calculations.
Logical Fault	This category is a bit broad and is meant to cover implementations that contain flaws in the way they are implemented, either due to unimplemented functionality, unaccounted-for edge cases or similar extraordinary scenarios.
Centralization Concern	This category covers all findings that relate to a significant degree of centralization present in the project and as such the potential of a Single-Point-of-Failure (SPoF) for the project that we urge them to re-consider and potentially omit.
Reentrant Call	This category relates to findings that arise from re-entrant external calls (such as EIP-721 minting operations) and revolve around the inapplicacy of the Checks-Effects-Interactions (CEI) pattern, a pattern that dictates checks (`require` statements etc.) should occur before effects (local storage updates) and interactions (external calls) should be performed last.

APPENDIX

Severity Definition

In the ever-evolving world of blockchain technology, vulnerabilities continue to take on new forms and arise as more innovative projects manifest, new blockchain-level features are introduced, and novel layer-2 solutions are launched. When performing security reviews, we are tasked with classifying the various types of vulnerabilities we identify into subcategories to better aid our readers in understanding their impact.

Within this page, we will clarify what each severity level stands for and our approach in categorizing the findings we pinpoint in our audits.

> ! To note, all severity assessments are performed **as if the contract's logic cannot be upgraded** regardless of the underlying implementation.

Severity Matrix

There are five distinct severity levels within our reports; `unknown`, `informational`, `minor`, `medium`, and `major`. A TL;DR overview table can be found below as well as a dedicated chapter to each severity level:

	IMPACT NONE	IMPACT LOW	IMPACT MODERATE	IMPACT HIGH
LIKELIHOOD NONE	Informational	Informational	Informational	Informational
LIKELIHOOD LOW	Informational	Minor	Minor	Medium
LIKELIHOOD MODERATE	Informational	Minor	Medium	Major
LIKELIHOOD HIGH	Informational	Medium	Major	Major

Severity Levels

Unknown Severity

The `unknown` severity level is reserved for misbehaviors we observe in the codebase that cannot be quantified using the above metrics. Examples of such vulnerabilities include potentially desirable system behavior that is undocumented, reliance on external dependencies that are out-of-scope but could result in some form of vulnerability arising, use of external out-of-scope contracts that appears incorrect but cannot be pinpointed, and other such vulnerabilities.

In general, `unknown` severity level vulnerabilities require follow-up information by the project being audited and are either adjusted in severity (if valid), or marked as nullified (if invalid).

Additionally, the `unknown` severity level is sometimes assigned to centralization issues that cannot be assessed in likelihood due to their exploitation being tied to the honesty of the project's team.

Informational Severity

The `informational` severity level is dedicated to findings that do not affect the code functionally and tend to be stylistic or optimizational in nature. Certain edge cases are also set under `informational` vulnerabilities, such as overflow operations that will not manifest in the lifetime of the contract but should be guarded against as a best practice, to give an example.

Minor Severity

The `minor` severity level is meant for vulnerabilities that require functional changes in the code but tend to either have little impact or be unlikely to be recreated in a production environment. These findings can be acknowledged except for findings with a moderate impact but low likelihood which must be alleviated.

Medium Severity

The `medium` severity level is assigned to vulnerabilities that must be alleviated and have an observable impact on the overall project. These findings can only be acknowdged if the project deems them desirable behavior and we disagree with their point-of-view, instead urging them to reconsider their stance while marking the exhibit as acknowledged given that the project has ultimate say as to what vulnerabilities they end up patching in their system.

Major Severity

The `major` severity level is the maximum that can be specified for a finding and indicates a significant flaw in the code that must be alleviated.

Likelihood & Impact Assessment

As the preface chapter specifies, the blockchain space is constantly reinventing itself meaning that new vulnerabilities take place and our understanding of what security means differs year-to-year.

In order to reliably assess the likelihood and impact of a particular vulnerability, we instead apply an abstract measurement of a vulnerability's impact, duration the impact is applied for, and probability that the vulnerability would be exploited in a production environment.

Our proposed definitions are inspired by multiple sources in the security community and are as follows:

IMPACT

HIGH

A core invariant of the protocol can be broken for an extended duration.

MODERATE

A non-core invariant of the protocol can be broken for an extended duration or at scale, or an otherwise major-severity issue is reduced due to hypotheticals or external factors affecting likelihood.

LIKELIHOOD

HIGH

A flaw in the code that can be exploited trivially and is ever-present.

MODERATE

A flaw in the code that requires some external factors to be exploited that are likely to manifest in practice.

LOW

A non-core invariant of the protocol can be broken with reduced likelihood or impact.

NONE

A code or documentation flaw whose impact does not achieve low severity, or an issue without theoretical impact; a valuable best-practice

LOW

A flaw in the code that requires multiple external factors to be exploited that may manifest in practice but would be unlikely to do so.

NONE

A flaw in the code that requires external factors proven to be impossible in a production environment, either due to mathematical constraints, operational constraints, or system-related factors (i.e. EIP-20 tokens not being re-entrant).

APPENDIX

Disclaimer

The following disclaimer applies to all versions of the audit report produced (preliminary / public / private) and is in effect for all past, current, and future audit reports that are produced and hosted under Omniscia:

IMPORTANT TERMS & CONDITIONS REGARDING OUR SECURITY AUDITS/REVIEWS/REPORTS AND ALL PUBLIC/PRIVATE CONTENT/DELIVERABLES

Omniscia ("Omniscia") has conducted an independent security review to verify the integrity of and highlight any vulnerabilities, bugs or errors, intentional or unintentional, that may be present in the codebase that were provided for the scope of this Engagement.

Blockchain technology and the cryptographic assets it supports are nascent technologies. This makes them extremely volatile assets. Any assessment report obtained on such volatile and nascent assets may include unpredictable results which may lead to positive or negative outcomes.

In some cases, services provided may be reliant on a variety of third parties. This security review does not constitute endorsement, agreement or acceptance for the Project and technology that was reviewed. Users relying on this security review should not consider this as having any merit for financial advice or technological due diligence in any shape, form or nature.

The veracity and accuracy of the findings presented in this report relate solely to the proficiency, competence, aptitude and discretion of our auditors. Omniscia and its employees make no guarantees, nor assurance that the contracts are free of exploits, bugs, vulnerabilities, deprecation of technologies or any system / economical / mathematical malfunction.

This audit report shall not be printed, saved, disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Omniscia.

All the information/opinions/suggestions provided in this report does not constitute financial or investment advice, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report.

Information in this report is provided 'as is'. Omniscia is under no covenant to the completeness, accuracy or solidity of the contracts reviewed. Omniscia's goal is to help reduce the attack vectors/surface and the high level of variance associated with utilizing new and consistently changing technologies.

Omniscia in no way claims any guarantee, warranty or assurance of security or functionality of the technology that was in scope for this security review.

In no event will Omniscia, its partners, employees, agents or any parties related to the design/creation of this security review be ever liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this security review.

Cryptocurrencies and all other technologies directly or indirectly related to cryptocurrencies are not standardized, highly prone to malfunction and extremely speculative by nature. No due diligence and/or safeguards may be insufficient and users should exercise maximum caution when participating and/or investing in this nascent industry.

The preparation of this security review has made all reasonable attempts to provide clear and actionable recommendations to the Project team (the “client”) with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts in scope for this engagement.

It is the sole responsibility of the Project team to provide adequate levels of test and perform the necessary checks to ensure that the contracts are functioning as intended, and more specifically to ensure that the functions contained within the contracts in scope have the desired intended effects, functionalities and outcomes, as documented by the Project team.

All services, the security reports, discussions, work product, attack vectors description or any other materials, products or results of this security review engagement is provided "as is" and "as available" and with all faults, uncertainty and defects without warranty or guarantee of any kind.

Omniscia will assume no liability or responsibility for delays, errors, mistakes, or any inaccuracies of content, suggestions, materials or for any loss, delay, damage of any kind which arose as a result of this engagement/security review.

Omniscia will assume no liability or responsibility for any personal injury, property damage, of any kind whatsoever that resulted in this engagement and the customer having access

to or use of the products, engineers, services, security report, or any other other materials.

For avoidance of doubt, this report, its content, access, and/or usage thereof, including any associated services or materials, shall not be considered or relied upon as any form of financial, investment, tax, legal, regulatory, or any other type of advice.

© 2026 Omniscia · revision 69e22d825b · ❖ 0a0d41ab14